Actividad 3: Enfoque Modular y Diseño Top-Down - LPR 5° 3° A-B

Leon Melgarejo

E.E.S.T. N.º 1 "Eduardo Ader" Vicente López

Curso: Laboratorio de Programación 5°

Profesor: Mansilla Muñoz York E.

Fecha: 17/09/2026

Este trabajo tiene como objetivo presentar la estructura general de un informe académico siguiendo las normas del estilo APA (7.ª edición). Aquí se desarrollarán los contenidos trabajados durante la actividad, incluyendo descripciones, análisis y conclusiones sobre el uso de entornos de desarrollo y conceptos básicos de programación.

De forma específica, esta entrega profundiza en la aplicación de un enfoque estructurado y la metodología de diseño Top-Down (de lo general a lo particular). A través de este paradigma, se aborda la descomposición de problemas complejos de ingeniería mediante la creación de funciones modulares en lenguaje C++, aplicadas de forma práctica en un algoritmo de cálculo geométrico y simulación gráfica en consola.

1. El Enfoque de Diseño Top-Down y Modularización

El diseño Top-Down consiste en tomar un problema complejo de ingeniería y dividirlo de manera sucesiva en módulos o subprogramas más pequeños, simples y fáciles de resolver de forma independiente. En el ámbito del lenguaje C++, este planteamiento se implementa de manera directa mediante la declaración y definición de **Funciones** creadas por el desarrollador.

De acuerdo con las bases del taller, la adopción de una arquitectura modular provee tres ventajas operativas críticas en el ciclo de vida del software:

Optimización de la Depuración: Facilita la localización y el aislamiento de errores lógicos o fallos sintácticos de manera rápida dentro de bloques específicos.

Eficiencia del Código: Fomenta la reutilización de rutinas computacionales, evitando la duplicación innecesaria de líneas de instrucciones en el programa.

Organización del Trabajo en Equipo: Permite la división ordenada de tareas de desarrollo entre múltiples programadores bajo interfaces claras.

2. Glosario Técnico Institucional

Diseño Top-Down: Metodología que analiza y descompone un sistema jerárquicamente desde el nivel abstracto general hacia componentes especializados e independientes.

Modularizar: Técnica de desarrollo de software orientada a fraccionar un programa en subprogramas funcionales independientes llamados funciones, mejorando la legibilidad y el mantenimiento.

Prototipo de Función: También conocido como "declaración de firma", se ubica antes del bloque principal `main()`. Su rol es notificar formalmente al compilador el identificador de la función, su tipo de retorno y la lista de parámetros requeridos para su invocación.

Retorno (Return): El valor, variable o resultado computado final que una función devuelve de manera explícita al contexto del programa invocador (como el bloque `main`) al concluir sus operaciones internas.

Constante (const): Directiva de almacenamiento que bloquea un casillero o dirección de memoria RAM de forma inmutable, impidiendo la alteración de su valor asignado durante toda la ejecución del software (por ejemplo, el valor matemático de π).

3. Organización de Archivos y Repositorio Git

El proyecto se estructuró de manera estricta bajo la nomenclatura definida en el laboratorio. La raíz del directorio se denominó `/calculoarea/` y se organizó internamente de la siguiente forma:

`/docs/`: Aloja la documentación formal del proyecto bajo el nombre obligatorio `InformeProyectoCALCULOAREA.pdf`.

`/src/`: Carpeta técnica exclusiva para el código fuente modular, conteniendo el fichero central `main.cpp`.

`/capturas/`: Espacio reservado para resguardar las evidencias gráficas locales del funcionamiento: `compilacion_area.png` y `diagrama_flujo.png`.

Ficheros de control: Inclusión en la raíz de los archivos operativos `.gitignore`, `README.md` y `LICENSE` (Licencia MIT).

4. Resolución del Reto Práctico: Código Modular Completo

A continuación, se detalla el código completo implementado en `main.cpp`, integrando las líneas solicitadas para la invocación del módulo, la salida formateada del área y el desarrollo de la lógica matemática de la función:

cpp

```
/*
   ================================================================

   Materia: Laboratorio de Programación (LPR)

   E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

   Curso: 5° Año

   Profesor: Prof. York

   Estudiante: Lopilato Luisana

   Archivo: main.cpp

   Objetivo: Calcular el area de un circulo mediante funciones
modulares

             y simular una diana de tiro en ASCII.

   ================================================================

*/
```

```
#include <iostream>


using namespace std;


// ========================================================================

// 1. PROTOTIPO DE LA FUNCIÓN (Firma de la función)

// Le avisamos al compilador la existencia del módulo de cálculo antes
del main

// ========================================================================

double calcularAreaCirculo(double radio);


int main() {

    double radioEstudiante;


    //
========================================================================

    // REGLA OBLIGATORIA (EVITA EL PLAGIO):

    // Modifiquen la salida de pantalla agregando su Nombre y Apellido
reales.

    //
========================================================================

    cout << "======================================================" <<
endl;

    cout << "  CALCULADORA DE AREA MODULAR - ESTUDIANTE: Luisana
Lopilato" << endl;

    cout << "======================================================" <<
endl;


    cout << "=> Ingrese el radio del circulo/diana (en cm): ";
```

```
    cin >> radioEstudiante;



    // FILTRO DE CONSISTENCIA (Validación del dato geométrico)

    if (radioEstudiante <= 0) {

        cout << "[ERROR] El radio debe ser un valor positivo y mayor a
cero." << endl;

        return 1; // Salida controlada del sistema indicando error en
consola

    }



    //
==========================================================================

    // 2. LLAMADA / INVOCACIÓN DE LA FUNCIÓN (COMPLETAR)

    // Convoquen al módulo 'calcularAreaCirculo' pasándole
'radioEstudiante'

    // y guarden el resultado en una nueva variable decimal llamada
'areaFinal'.

    //
==========================================================================



    double areaFinal = calcularAreaCirculo(radioEstudiante);



    //
==========================================================================

    // 3. SALIDA DE DATOS (COMPLETAR)

    // Muestren en pantalla el área calculada utilizando 'areaFinal'.

    //
==========================================================================
```

```
    cout << "=> El area calculada de la diana es: " << areaFinal << "
cm2." << endl;



    //
==========================================================================

    // BONUS GAMIFICACIÓN: RENDER DE LA DIANA EN CONSOLA

    // Dependiendo del radio que ingresaron, la consola simulará un
blanco

    // de tiro correspondiente al tamaño calculado.

    //
==========================================================================

    cout << "\n[SISTEMA] Dibujando escala de la Diana en la RAM..." <<
endl;

    if (radioEstudiante <= 5.0) {

        cout << "       .---.       " << endl;

        cout << "      /  X  \\     -> [DIANA MINI / COMPACTA]" <<
endl;

        cout << "      \\  *  /     " << endl;

        cout << "       '---'       " << endl;

    } else if (radioEstudiante <= 12.0) {

        cout << "       .---.       " << endl;

        cout << "     / .---. \\     " << endl;

        cout << "    | /  O  \\ |   -> [DIANA ESTÁNDAR DE TIRO]" <<
endl;

        cout << "    | \\  *  / |    " << endl;

        cout << "     \\ '---' /     " << endl;

        cout << "       '---'       " << endl;

    } else {

        cout << "       .---.       " << endl;
```

```
        cout << "     / .---. \\     " << endl;
        cout << "    | / .-. \\ |    " << endl;
        cout << "    | |  X  | |   -> [DIANA GIGANTE DE COBERTURA]" <<
endl;
        cout << "    | \\ '-' / |    " << endl;
        cout << "     \\ '---' /     " << endl;
        cout << "       '---'       " << endl;
    }

    cout << "=======================================================" <<
endl;
    return 0; // Finalización exitosa de Windows (Código 0)
}

// ========================================================================
// 4. DEFINICIÓN / DESARROLLO DE LA FUNCIÓN (COMPLETAR)
// Implementen la fórmula matemática de la geometría utilizando
variables
// locales y la constante de pi locked en memoria.
// ========================================================================
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;

    double area = PI * r * r;
    return area;
}
```

Usa el código con precaución.

Comandos Ejecutados en PowerShell (Windows):

1. **Compilación con ejecutable personalizado:** `g++ src/main.cpp -o src/area.exe`
2. **Ejecución de la aplicación:** `.\src\area.exe`

En esta sección se resumen las principales ideas trabajadas y se expresan opiniones personales sobre lo aprendido en la actividad. Es importante incluir reflexiones sobre el uso de herramientas y el inicio en la programación.

El diseño Top-Down y la implementación de funciones demuestran ser pilares clave para escribir código limpio, legible y escalable en la formación técnico-profesional. Como complemento analítico de la práctica, se da respuesta formal al **Cuestionario Técnico de Control** incorporado por la cátedra:

> **Pregunta 1: El Prototipo de la Función**
> *Pregunta:* Si borráramos la línea 15 (`double calcularAreaCirculo(double radio);`), ¿por qué creés que el compilador no sabría qué hacer cuando intente leer la función `main`?
> *Respuesta:* El compilador de C++ realiza una lectura lineal descendente (de arriba hacia abajo) del archivo de código. Si se elimina el prototipo, al ingresar al bloque `main()` e identificar la invocación de `calcularAreaCirculo`, el compilador no poseerá registro previo de su existencia, de su tipo de retorno ni de sus parámetros requeridos, interrumpiendo el proceso con un error sintáctico de identificador no declarado.

> **Pregunta 2: El Filtro de Consistencia**
> *Pregunta:* ¿Qué problema físico o geométrico ocurriría si intentáramos calcular el área de un círculo con un radio de 0 cm o de -5 cm? ¿Por qué es importante controlar esto?
> *Respuesta:* Desde la perspectiva de la geometría euclidiana, un radio igual a cero representa un punto dimensional (área nula) y un radio con valor negativo carece por completo de sentido físico o espacial real. Es fundamental implementar filtros de consistencia mediante estructuras condicionales para blindar el programa contra ingresos erróneos de los usuarios, previniendo cálculos matemáticos ilógicos y garantizando la confiabilidad del sistema antes de procesar datos en las funciones de cálculo.

Pregunta 3: Proteger Constantes en RAM
Pregunta: ¿Por qué considerás que es más seguro para un sistema crítico (como el software de un avión o un sistema médico) declarar valores como "inmutables" (`const`) en lugar de usar variables comunes?
Respuesta: En sistemas de alta criticidad, donde un fallo de software puede comprometer vidas humanas, el uso de la palabra clave `const` proporciona seguridad a nivel de hardware y compilación. Al declarar un valor inmutable, el compilador bloquea de manera estricta esa dirección de memoria RAM. Si por un error lógico en el código, un desbordamiento de búfer o una instrucción corrupta se intenta alterar dicho valor en tiempo de ejecución, el compilador detendrá inmediatamente el proceso por seguridad, mitigando accidentes drásticos derivados de parámetros alterados.

E.E.S.T. N.º 1 "Eduardo Ader". (2026). *Actividad 3: Enfoque Modular y Diseño Top-Down - LPR 5° 3° A-B (Presencial/Virtual Asincrónico)*. Laboratorio de Programación, Vicente López, Buenos Aires.

Mansilla Muñoz, Y. E. (2026). *Glosario de Conceptos Técnicos y Criterios de Evaluación en Entornos Híbridos*. E.E.S.T. N.º 1, Vicente López, Buenos Aires.

https://github.com/letan-doto07/actividad-3-enfoque-modular-y-dise-o-top-down-